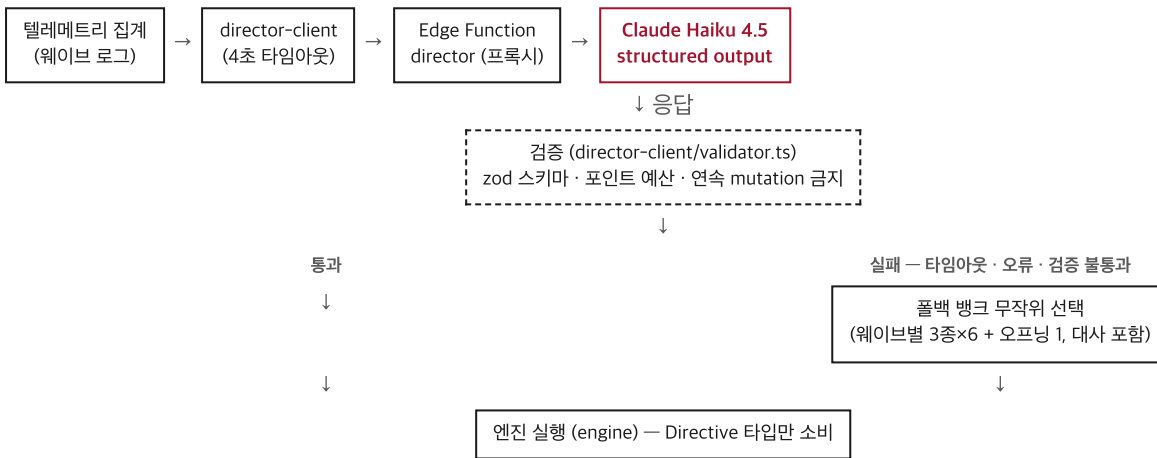


이 게임은 두 층위에서 AI를 쓴다. 게임 안에는 플레이어의 로그를 읽고 다음 웨이브를 설계하는 AI 디렉터가 있고, 게임을 만드는 과정 자체도 스펙 → 플랜 → 구현 → 리뷰로 이어지는 AI 파이프라인으로 진행했다. 이 문서는 두 층을 순서대로 설명한다 — [1층] 게임 안의 AI, [2층] 개발에 쓴 AI. 마지막에 외부 에셋·오픈소스 출처를 정리한다.

[1층] 게임 안의 AI — 디렉터

1.1 아키텍처

디렉터는 게임을 자유롭게 조작하지 않는다. 매 웨이브 종료 시 정해진 파이프라인을 한 번 통과해 다음 웨이브의 디렉티브 하나를 만들어낼 뿐이다. 정상 경로와 장애 경로 모두 같은 엔진 실행부로 합류한다.



엔진은 그 디렉티브가 LLM에서 왔는지 폴백 बैं크에서 왔는지 모른다. 계약은 `src/contracts/directive.ts` 하나(zod 스키마)이고, Edge Function은 별도 번들이라 소스를 import할 수 없어 같은 스키마의 수동 사본을 JSON Schema 형식으로 들고 있다 — 계약을 바꾸면 두 곳을 함께 고쳐야 한다.

1.2 입출력 계약

director-client가 웨이브 종료마다 집계해 보내는 플레이 로그:

```
{
  "wave": 2, "clearTimeSec": 34.2, "hpLost": 2,
  "damageSources": {"chaser": 1, "shooter": 1},
  "movement": {"quadrantTime": {"NW": 0.4, "NE": 0.1, "SW": 0.4, "SE": 0.1},
    "wallHugRatio": 0.55, "dashCount": 6},
  "combat": {"kills": {"chaser": 12, "shooter": 3, "splitter": 2}, "accuracy": 0.61},
  "upgrades": ["DAMAGE_UP", "MOVE_SPEED_UP"],
  "prevMutations": ["FOG"]
}
```

디렉터가 돌려주는 디렉티브 — 자유 텍스트가 아니라 이 네 필드로 제한된 JSON:

```
{
  "composition": [
    { "type": "chaser|shooter|splitter", "count": 1, "spawn": "N|S|E|W|RING|Pincer|BEHIND", "elite": false }
  ],
  "mutation": "NONE|LAVA_LEFT|LAVA_RIGHT|FOG|SPEED_SURGE|SHRINK_ARENA|SPAWN_STORM",
  "taunt": "60자 이내 — 로그의 실제 습관 1개 지목",
  "intent": "100자 이내 — 설계 의도"
}
```

1.3 시스템 프롬프트 전문

Edge Function이 Claude Haiku 4.5에 보내는 시스템 프롬프트 두 개를 그대로 옮긴다. 하나는 매 웨이브의 디렉티브 생성용, 하나는 런 종료 시 리포트 생성용이다.

디렉티브 생성 (mode: 'directive')

너는 아케이드 게임의 'AI 디렉터'다. 플레이어의 웨이브 로그를 읽고 다음 웨이브를 설계한다.

규칙:

- composition 총 비용(chaser 1, shooter 2, splitter 2, elite는 $\times 3$)은 예산 이하로. 예산은 사용자 메시지에 준다.
- taunt는 한국어 60자 이내. 반드시 로그에서 실제로 관찰되는 습관 하나를 꼭 집어 지목하라(예: 벽 붙기 wallHugRatio, 특정 사보면 체류, 낮은 명중률, 대시 남용, 특정 적에게 반복 피격, 업그레이드 성향). 그리고 설계가 그 습관을 실제로 공략해야 한다.
- 해석 규칙: hpLost가 damageSources 합보다 크면 그 차이는 지형 피해(용암 존·축소 경계)다 - 지형에 자주 타는 플레이어에게는 그 습관을 지목할 수 있다.
- 직전 mutation과 같은 것은 고르지 마라.
- 어렵지만 이길 수 있게: 플레이어가 고전한 요소는 유지하되 물량으로 압살하지 마라.
- intent는 설계 의도 100자 이내.

사용자 메시지: 웨이브 $\${wave}$ 설계. 예산: $\${budget}$. 직전 mutation: $\${prevMutation}$. + 플레이 로그 JSON.

리포트 생성 (mode: 'report')

너는 게임의 AI 디렉터다. 방금 끝난 판의 전체 로그를 보고, 플레이어의 스타일을 분석하는 리포트를 한국어 400자 내외로 써라. 마지막 줄에 "칭호: <4~8자 칭호>" 형식으로 칭호를 붙여라. 관찰된 사실만 근거로, 디렉터의 시점(1인칭)으로, 패배시켰다면 여유롭게, 패배했다면 인정하며.

호출 파라미터: model `claude-haiku-4-5` max_tokens 500(디렉티브)/700(리포트). 디렉티브 호출은 `output_config: { format: { type: 'json_schema', schema: DIRECTIVE_JSON_SCHEMA } }`로 API 레벨에서 출력 스키마를 강제한다 — 응답 텍스트를 파싱해 JSON을 추출하는 방식이 아니라, 모델이 스키마에 맞는 JSON만 내도록 디코딩 단계에서 제약한다.

1.4 권한 경계 설계

디렉터에게 준 것은 코드가 아니라 어휘다. composition·mutation·taunt·intent 네 필드 바깥으로는 어떤 값도 낼 수 없고, 그 안에서도 API 스키마(서버)와 zod(클라이언트) 두 번을 거쳐야 엔진에 닿는다. 이렇게 설계한 이유는 하나다 — 심사자가 이 게임을 직접 플레이하는 동안 LLM 쪽에서 무슨 일이 나도 게임이 멈추면 안 된다.

층위	규칙
스키마	zod 검증 — 필드 누락·enum 밖 값·문자열 길이 초과 시 거부
예산	<code>budgetFor(wave) = 8 + wave$\times$4 + max(0, wave-5)$\times$12</code> 이하만 허용(비용: chaser 1·shooter 2·splitter 2, elite $\times 3$). 난이도 곡선을 LLM이 깨뜨릴 수 없다
반복	직전 mutation과 동일하면 검증 계층이 NONE으로 강제 교체
시간	4초 초과·네트워크 오류·검증 실패 → 사전 제작 풀백 बैं크(웨이브별 3종 $\times 6$ + 오프닝 1)에서 무작위 선택. 대사도 बैं크에 포함돼 있어 플레이어는 풀백 여부를 알 수 없다

이 경계는 실측으로 확인했다. 장애 경로(네트워크 오류·타임아웃·스키마 위반)는 유닛 테스트 4건에 더해, 배포 전 상태(프록시 URL 없음)에서 오프닝부터 웨이브7 WIN까지 전 구간을 풀백만으로 완주한 라이브 런으로 검증했다 — 웨이브 전환 중 정지는 없었다. 예산 규칙은 유닛 테스트 8건과, 풀백 बैं크에 들어있는 디렉티브 19개(오프닝 1 + 웨이브별 3종 $\times 6$) 전수 검증으로 확인했다. 적 58기가 동시에 활성화된 상태에서 프레임당 처리 비용은 평균 0.145ms(60fps 예산 16.7ms 대비 약 115배 여유)로, 디렉터가 물량을 늘려도 프레임이 무너질 여지는 작다. 세션당 호출 20회·일일 500회 캡은 경계값 시뮬레이션으로 확인했다(21회째부터 거부, 501회째부터 거부).

설계 사례 — 계약을 바꾸지 않고 프롬프트로 푼 문제

지형 피해 해석 규칙이 이 원칙을 보여준다. 텔레메트리는 적 타입별 피해(damageSources)만 집계하고 용암·축소 지형에 의한 피해는 별도 필드가 없다 — 계약을 바꾸면 엔진·검증기·뱅크 19개 항목을 전부 다시 맞춰야 한다. 대신 시스템 프롬프트에 해석 규칙 한 줄을 추가했다:

`hpLost -  $\Sigma$  damageSources = 지형 피해`. 계약은 그대로 두고, 디렉터가 로그에서 지형 피해를 스스로 읽어내게 했다.

[2층] 개발에 쓴 AI — AI-DLC 파이프라인

2.1 파이프라인 구조

게임 코드도 같은 원칙으로 만들었다 — AI에게 자유를 주는 대신 단계마다 문서로 경계를 그었다. 흐름은 스펙(frozen) → 플랜(승인) → 태스크별 구현 에이전트 → 독립 리뷰 에이전트 → fix 라운드 → 스코프 재리뷰 → 다음 태스크다. 구현 에이전트와 리뷰 에이전트는 분리돼 있다 — 코드를 쓴 에이전트가 자기 코드를 검사하지 않는다. 모든 태스크는 커밋 전에 하네스 (`scripts/ai_harness.sh --fast`: tsc+lint+유닛, `--full`: +빌드)를 통과해야 하고, 통과 여부는 `HARNESS RESULT: PASS|FAIL` 한 줄로 확정된다. 계약은 `src/contracts/directive.ts` 하나가 SSOT다 — 엔진은 이 타입만 알고 디렉터 내부(LLM인지 뱅크인지)를 모른다.

2.2 핵심 서사 — 코드보다 계획을 먼저 고친다

Never Vibe Code 원칙: 구현 중 계획의 결함이 드러나면 코드부터 고치지 않는다. 계획·스펙을 먼저 고쳐 커밋한 뒤에야 구현을 재개한다.

`plan:` / `spec:` / `docs:` 커밋이 그 기록이고, 각각 뒤따르는 구현 커밋과 짝을 이룬다.

발견	계획 커밋	구현 커밋	무엇이 잘못됐었나
예산 곡선 ↔ 피날레 콘텐츠 모순	<code>a2cca53</code>	<code>f250404</code>	계획의 예산식이 계획 자신의 웨이브 6~7 बैंक 콘텐츠들 거부
텔레메트리 픽스처가 두 지표를 판별 못함	<code>8c8acf6</code>	<code>397fb59</code>	벽면 체류·사분면 체류가 항상 같은 값이 되는 픽스처
원격 QA 제약(백그라운드 탭 rAF 정지)	<code>498c28e</code>	(Task 8 구현)	검증 수단 부재를 계획에 반영해 dev QA 혹 추가
지형 피해가 적 타입별 집계에 안 잡힘	<code>2bdc965</code>	(Task 7 프록시)	계약을 바꾸는 대신 프롬프트에 해석 규칙 추가
Phaser 버전 표기 오류(3→4)	<code>e15f9dc</code>	—	스펙·플랜의 사실 오류를 실측 후 정정

첫 사례 — 이 방식이 실제로 작동한다는 증거

웨이브가 진행될수록 예산이 커지도록 짠 곡선이, 같은 계획서가 정해둔 웨이브 6~7 बैंक 콘텐츠보다 낮았다 — 계획대로면 사전 제작해 둔 후반 웨이브 디렉티브를 배치할 예산이 부족했다. 구현 에이전트는 콘텐츠를 임의로 깎아 맞추지 않았다. 대신 정지하고 컨트롤러에게 보고했고, 컨트롤러가 계획의 예산 곡선을 개정해 커밋(`a2cca53`)한 뒤에야 구현이 재개돼 다음 커밋(`f250404`)으로 이어졌다. 구현 에이전트가 승인된 계획을 스스로 깎지 않고 멈춰 보고했다는 것 자체가 이 파이프라인의 설계다 — 코드 층위의 권한 경계(1.4절)를 개발 프로세스 층위에도 그대로 적용한 것이다.

2.3 리뷰가 잡은 실결함

태스크마다 별도의 리뷰 에이전트가 스펙 준수와 코드 품질을 판정했다. Critical-Important 지적은 fix 라운드로 처리하고 스코프 재리뷰로 달았다. 아래 7건 중 2건은 자동 테스트가 아니라 라이브 플레이 중에만 드러났다.

태스크	지적	성격
4	픽스처가 두 지표를 판별 못함	테스트가 검증하지 못하는 상태를 검증한다고 착각
5	엘리트 이속 상승 미구현	스펙 항목이 브리프 전달 과정에서 누락
6	인터벌 잔여 탄환이 마감된 웨이브 로그를 오염	라이브 레퍼런스·로테이션 타이밍
6	SPAWN_STORM 빈 배치로 클리어 최대 8초 지연	현 콘텐츠엔 미발현, LLM 디렉티브에서 발현 가능
7	Edge Function 스키마 수동 사본 동기화 의무 미문서화	다음 세션이 알 수 없는 암묵지
8	타이핑 스킵 후 잔여 타이머가 확정 대사를 덮어쓰	라이브 플레이로만 발견(구현자 자체 발견)
9	인터벌 대기 중 사망 시 웨이브 로그 중복 기록	라이브 플레이로만 발견(구현자 자체 발견)

2.4 규모

이 문서를 쓰는 시점(2026-08-05) 기준 실측치다. 제출 직전 최종 재산출 대상. 커밋 33개, TypeScript 26파일 2,824줄, 유닛 테스트 33개(6개 파일 전체 통과). 커밋 유형 분포: feat 10 · plan 5 · fix 7 · spec 3 · docs 5 · test 1 · chore 2. `plan` 커밋이 별도 카테고리 5건 존재한다는 사실 자

체가 이 파이프라인의 증거다 — 코드를 고치기 전에 계획 문서를 고친 기록이 커밋 히스토리에 그대로 남아 있다.

2.5 원격 QA 기법

자동화 브라우저는 백그라운드 탭에서 requestAnimationFrame이 스로틀돼 게임 시간이 흐르지 않는다 — 인터벌 타이머도, SPAWN_STORM 같은 예약 스폰도 실시간 대기로는 진행되지 않는다. `window.__game.loop.step(t)`를 고정 간격으로 직접 호출해 프레임을 결정론적으로 전진시키는 방식(`__pump(n)`)으로 우회했다. 이 기법으로 7웨이브 완주, 승/패 양 엔딩, 재시작 초기화를 원격으로 검증했다.

[공통] 외부 에셋 / 오픈소스 출처

이미지·사운드 파일은 리포지토리에 하나도 없다(png·jpg·svg·mp3·wav·ogg·폰트 파일 검색 결과 0건). 모든 그래픽은 Phaser의 `Graphics.generateTexture`로 런타임에 생성하고, 모든 효과음은 zzfx로 절차 생성한다. 외부 에셋 라이선스 리스크는 0이다. 이 프로젝트(DIRECTOR'S CUT) 자체도 MIT 라이선스로 공개한다(저장소 루트 `LICENSE` 파일).

패키지	버전	라이선스	역할
phaser	4.2.1	MIT	게임 엔진
zod	4.4.3	MIT	디렉티브 스키마 검증
zzfx	1.3.2	MIT	절차적 효과음 생성
typescript	7.0.2	Apache-2.0	개발 도구 — 타입체크·빌드(dev)
@types/node	26.1.2	MIT	타입 정의(dev)
vite	8.2.0	MIT	빌드·개발 서버(dev)
vitest	4.1.10	MIT	유닛 테스트 러너(dev)
@anthropic-ai/sdk	0.115.0(고정)*	MIT	Edge Function의 Claude API 클라이언트

* Edge Function은 Deno 런타임에서 `npm:@anthropic-ai/sdk` 스펙 임포트를 쓴다 — 프론트엔드 의존성처럼 lockfile로 관리되지 않아, 심사 기간(제출 8/10 ~ 본선 9/4~6) 중 상위 버전의 breaking change가 프록시를 깨뜨릴 위험을 막기 위해 정확 버전 (`npm:@anthropic-ai/sdk@0.115.0`)으로 고정했다.

AI 도구 사용 내역 — 개발 전 과정에서 쓴 AI 도구는 Claude Code(Claude Fable 5 / Opus 5)다. 스펙·플랜 수립, 구현·리뷰 에이전트 오케스트레이션에 사용했다. 게임 런타임에서 실제로 플레이어를 상대하는 AI는 Claude Haiku 4.5(Anthropic API, structured output)다.